

Conditional Statement and Loops

Matteo Fasiolo

Week 1 recap

Foundations

- Programs are instructions that a computer executes.
- A **compiler** translates C source code into machine code.

Edit, compile, run

- Write a .c file in VS Code, compile it with gcc, then run it.
- Use compiler messages and the debugger to investigate problems.

Core C

- `main` is the entry point; statements normally execute in order.
- Variables have types; expressions produce values; assignments store them.
- `printf` uses format specifiers such as `%d`, `%f` and `%s`.

Functions

- A function has typed parameters, a return type and a body.
- Functions are defined and called; variables inside them are local to that function.
- Calls can be nested: one function's output becomes another's input.

Tutorial: Input of a function can be the output of another.

The input of a **function** can be **any expression** whose value is **compatible with the input type**.

```
#include <stdio.h>

double square(double a){
    return a*a;
}

int main(void){

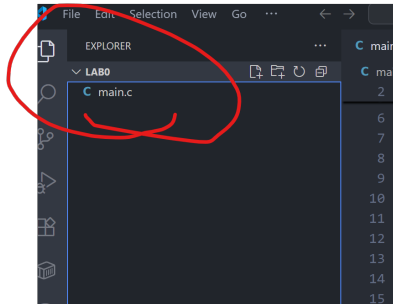
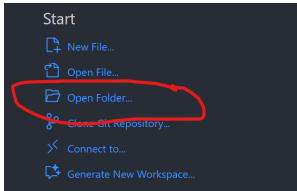
    double a = 2;

    printf("%f", square(square(a)));

    return 0;
}
```

Issues in Previous Lab

Use “Open Folder” to open the folder that contains your .c file!



Today's focus: Control-flow

Normally, statements in a C program execute from top to bottom

```
#include <stdio.h>
int main(void){
    double a = 2;
    double b = 4;
    double c = a * b;
    printf("The result is %f\n", c);
    return 0;
}
```

However, this is not always the case.

```
#include <stdio.h>

double square(double a){
    return a*a;
}

int main(void){

    double a = 2;

    printf("%f", square(square(a)));

    return 0;
}
```

Here return a*a; executes (twice) before return 0;

Let's look at double_square.c

Hence function calls can alter the order in which code executes.

The order in which a program executes its instructions is called **control flow**.

The default is **sequential** flow: Top to bottom.

The control flow can be changed by:

- A **function call** moves execution to a function and then returns.
- A **conditional statement** chooses which instructions to execute.
- A **loop** repeats a group of instructions.

Today we will focus on the conditional statements and loops.

Conditional Statement

In many cases, we want to write a program responding to different conditions:

- If a score ≥ 40 , print “pass!”. Otherwise, print “fail”.
- If user's input equals the password, proceed to log in. Otherwise, print out an error message.

This simple conditional statement is called “if-else” statement and exists in many programming languages.

An if-else statement in C is written as follows:

```
if (condition){  
    // statements to be executed, if condition is true  
} else {  
    // statements to be executed, if condition is false  
}
```

Conditions are often built using relational and logical operators.

Example:

```
int score = 41;
if(score >= 40){
    printf("pass!\n");
}else{
    printf("fail!\n");
}
// prints "pass"
```

The else clause is optional.

```
int password = 4321;
if(password == 1234){
    printf("password correct!\n");
}
printf("1...");
printf("2...\n");
// Prints out 1... 2...
```

Relational and Logical Expressions

We often use **relational** and **logical expressions** as conditions in if-else statements.

For example

```
if(x>3){
```

where

- $x > 3$ is a relational **expression**
- $>$ is a relational **operator** that compares two **operands**
- the result of the expression is either **0 (false)** or **1 (true)**

Relational Operators:

- $>$ strictly greater: $2 > 1$ is TRUE, $2 > 2$ is FALSE
- $>=$ greater or equal: $2 >= 2$ is TRUE.
- $==$ equal to: $1 == 1$ is TRUE.
- $!=$ not equal: $2 != 1$ is TRUE.

NOTE, single $=$ is the assignment operator:

```
x = 3;
```

which is different from $x == 3$.

Confusing $==$ and $=$ is a source of sneaky bugs (same in R)!

Example:

```
#include <stdio.h>

int main(void){

    int x = 1;
    int y = 2;

    if(y = x){
        printf("y = %d \n", y);
    }

    return 0;
}
```

Let's run `logic_example.c` with the debugger in VSCode...

Not what we wanted!

Logical operators:

- `&&` is logical AND:
 - $x > 3 \ \&\& \ x < 10$ is true if $3 < x < 10$.
- `||` is logical OR:
 - $x_1 > 3 \ || \ x_2 > 3$ is true if $x_1 > 3$ or $x_2 > 3$.
- `!` is logical NOT:
 - $!(x > 3)$ is true if $x \leq 3$.

Note that:

- `&&` and `||` are **binary** (two operands)
- `!` is **unary** (one operand)
- `&&` and `||` are different from `&` and `|`.

Nested if-else statements and the if-else-if ladder

What if we have more than two branches?

For example, we classify students into 5 categories:

- $\text{score} \geq 70$, first class.
- $60 \leq \text{score} < 70$, two-one
- $50 \leq \text{score} < 60$, two-two
- $40 \leq \text{score} < 50$, pass
- $\text{score} < 40$, fail.

Can we use conditional statements to do that?

Useful property of if-else statements: They can be **nested**.

Example:

```
if(condition1){  
    // statements1  
} else {  
    if(condition2){  
        // statements2  
    } else {  
        // statements3  
    }  
}
```

The nesting can be arbitrarily deep.

Using nested if-else statements, we can write:

```
int score = 55;
if (score >= 70){
    printf("First Class.\n");
} else {
    if(score >= 60){ // why not 60 <= score < 70?
        printf("Two-One.\n");
    } else {
        if(score >= 50){
            printf("Two-Two.\n");
        } else {
            if(score >= 40){
                printf("Pass.\n");
            } else {
                printf("Fail.\n");
            }
        }
    }
}
} // Pretty hard to read, right?
```


If-else in C can be written:

```
if(condition){  
    statements1  
} else {  
    statements2  
}
```

where:

- a statement is a block of code, e.g. `printf("Hello!");`
- curly braces are used to bundle together multiple statements

If `statements1` or `statements2` contains only one statement, we can remove the braces:

```
if(condition) statement1; else statement2;
```

NOTE: code indentation does not have any effect in C.

This leads to the **if-else-if ladder**:

```
if (condition1){  
    statements1;  
} else if(condition2){  
    statements2;  
} else if(condition3){  
    statements3;  
} else {  
    statements4;  
}
```

because nested `if(condition2)` is one statement. Note:

- The program will check conditions **sequentially**.
- Once a true condition is found
 - It executes the associated statements and bypasses the rest.
- If none of the conditions are true, it executes the `else` statements (if there is one).

The ladder below prints out the classification given a score:

```
int score = 55;
if (score >= 70){
    printf("First Class.\n");
} else if(score >= 60){
    printf("Two-One.\n");
} else if(score >= 50){
    printf("Two-Two.\n");
} else if(score >= 40){
    printf("Pass.\n");
} else{
    printf("Fail\n");
}
```

Let's look at `if_else_if.c`.

Note that if an earlier condition check is true, it bypasses the entire ladder without checking the latter conditions!

If I made a mistake on the second condition:

```
int score = 55;
if (score >= 70){
    printf("First Class.\n");
} else if(score >= 50){
    printf("Two-One.\n");
} else if(score >= 50){
    printf("Two-Two.\n");
} else if(score >= 40){
    printf("Pass.\n");
}
```

Will it still print “two-two”?

The program runs, but the output is wrong!

Loops

In programming language, we sometimes want to repeat a certain operation many times.

Examples:

- Adding up a sequence of numbers
- Read a text file until it reaches the last line.

This mechanism is called a **loop**.

When it encounters a loop, the CPU will continue to execute a code block, until certain exit conditions are met.

A loop is another tool to alter the control flow.

While Loop

The simplest loop is while-loop and its syntax is:

```
while(condition){  
    statements  
}
```

The statements inside the braces will be run repeatedly as long as the condition is true. Example:

```
// print out all positive integer up to 10  
int i = 1;  
while(i <= 10){  
    printf("%d\n", i);  
    i = i + 1;  
}
```

Let's look at whileloop.c.

```
int i = 1;
while( i <= 10 ){
    printf("%d\n", i);
    i = i + 1;
}
```

Each repetition of the loop is called **iteration**.

- The loop above **iterates** 10 times.

In a loop, we commonly have an integer variable keeping the count of repetitions.

Such a variable is called **loop counter**:

- *i* is the loop counter in the above loop.
- The counter is initialized before the loop and is increased by one before the end of each iteration.
- We stop the loop by checking if $i \leq 10$.

For Loop

The for loop is another type of loop mechanism in C.

```
for(init statement; condition; update statement){  
    statements to be repeated  
}
```

Steps:

1. Initialise a counter.
2. Check condition:
 - If it is satisfied, go to step 3.
 - If not, exit the loop.
3. Run statements
4. Update counter and go back to 2.

The same operation as the while loop, but in one line.

Prints out all positive integer smaller than 10:

```
for(int i = 1; i <= 10; i = i + 1){  
    printf("%d\n", i);  
}
```

It is more compact than the while loop:

```
int i = 1;  
while(i <= 10){  
    printf("%d\n", i);  
    i = i + 1;  
}
```

Counter *i* initialized, checked and updated all in one line.

Let's look at `forloop.c`.

for vs while loop

Both loop mechanisms are the same:

- **Initialize, Check, Run statements, Update!**

Use **while** loop when **you do not know how many times the loop will be run.**

- When asking user's input, you do not know when the user will finish.
- When playing a game, you do not know when user will hit the "exit" button.

Use **for** loop when **you know exactly how many times the loop will repeat.**

- Print out number from 1-10.
- Sum up students' scores in a class.

Conclusion

Today, we learned how to determine the control flow using:

- Conditional Statements
 - if-else
 - if-else-if ladder
- Relational and Logical operators.
 - >, >=, ==, !=, &&, ||, !
- Loops
 - while loop
 - for loop